LeetCode Comprehensive Revision Cheatsheet

Core Fundamentals

1. Sliding Window

When to Use

- Contiguous subarray/substring
- Find optimal window (min/max length)
- Can expand/shrink based on conditions

Two Types

Fixed Size:
- Window size constant (k)
- Slide by adding right, removing left
- Example: Max Average Subarray

Variable Size:
- Window grows/shrinks on condition
- Expand: Add right element
- Shrink: Remove left until valid
- Examples: Min Window, Longest Substring

```
// Template
left = 0
for right in range(n):
    window.add(arr[right])

    while (condition_violated):
        window.remove(arr[left])
        left++

    result = update(result, window)
```

2. Two Pointers

Critical Question

Which pointer to move?

Strategies

Strategy 1: Move based on comparison
→ Compare elements at l and r
→ Move based on which needs change
→ Example: Container With Most Water
Strategy 2: Move based on running max/min
→ Track leftMax and rightMax
→ Move pointer with smaller max
→ Example: Trapping Rain Water

```
left = 0, right = n - 1

while left < right:
    // Calculate result

    if (condition):
        left++
    else:
        right--
```

3. Binary Search

Critical Choice

Loop condition determines approach!

left < right

✓ Finding a POSITION
✓ Answer guaranteed to exist
✓ Use: `right = mid`
✓ Exit: `left == right`
✓ Return: `nums[left]`

left <

✓ Finding EXACT VALUE
✓ Answer may not exist
✓ Use: `right = mid - 1`
✓ Exit: `left > right`
✓ Return: index or -1

Rule of Thumb

If `right = mid` → use `left < right`
If `right = mid - 1` → use `left <= right`

4. HashMap Optimization

HashMap to Array

When to optimize:
- Character problems (ASCII/lowercase)
- Limited value range
- Need O(1) space instead of O(k)

HashMap	Array
O(k) space	O(1) space
Flexible	ASCII only
Slower	Faster

```
// Conversion
map.get(c) -> arr[c - 'a']  // lowercase
map.get(c) -> arr[c - 'A']  // uppercase
map.get(c) -> arr[c]        // ASCII
```

Problem Solutions

1. Product of Array Except Self

Problem

Return array where output[i] = product of all elements except nums[i]
Complexity: O(n) time, O(1) space

Key Insight

Use prefix and suffix products
Build prefix in result, multiply suffix in-place

```java
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];

    // Build prefix products
    result[0] = 1;
    for (int i = 1; i < n; i++) {
        result[i] = result[i-1] * nums[i-1];
    }

    // Multiply by suffix products
    int suffix = 1;
    for (int i = n-1; i >= 0; i--) {
        result[i] *= suffix;
        suffix *= nums[i];
    }

    return result;
}
```

Common Mistakes

× Can't use division (zeros!)
× Don't create prefix/suffix arrays
✓ Build prefix in result, multiply suffix

2. Subarray Sum Equals K

Problem

Count subarrays with sum = k
Complexity: O(n) time, O(n) space

Key Insight

Prefix sum + HashMap to track frequencies
Formula: if prefixSum[j] - prefixSum[i] = k
then prefixSum[i] = prefixSum[j] - k

```java
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1);  // Base case

    int sum = 0, count = 0;

    for (int num : nums) {
        sum += num;

        if (prefixCount.containsKey(sum - k)) {
            count += prefixCount.get(sum - k);
        }

        prefixCount.put(sum,
            prefixCount.getOrDefault(sum, 0) + 1);
    }

    return count;
}
```

Common Mistakes

× Don't forget: `map.put(0, 1)`
× Can't use sliding window (negatives!)
✓ HashMap lets us check (sum - k)

3. Top K Frequent Elements

Problem

Find k most frequent elements
Complexity: O(n) time, O(n) space

Key Insight

Bucket sort by frequency
Frequency range: [1, n] → need n+1 buckets

```java
public int[] topKFrequent(int[] nums, int k) {
    // Count frequencies
    Map<Integer, Integer> count = new HashMap<>();
    for (int num : nums) {
        count.put(num, count.getOrDefault(num, 0) + 1);
    }

    // Bucket sort
    List<Integer>[] buckets = new List[nums.length + 1];
    for (int num : count.keySet()) {
        int freq = count.get(num);
        if (buckets[freq] == null) {
            buckets[freq] = new ArrayList<>();
        }
        buckets[freq].add(num);
    }

    // Collect top k
    int[] result = new int[k];
    int idx = 0;
    for (int i = buckets.length - 1; i >= 0 && idx < k; i--) {
        if (buckets[i] != null) {
            for (int num : buckets[i]) {
                result[idx++] = num;
                if (idx == k) return result;
            }
        }
    }

    return result;
}
```

Common Mistakes

× Don't use PriorityQueue O(n log k)
✓ Bucket sort is O(n) - optimal
✓ Array of ArrayLists, NOT LinkedList

4. Valid Parentheses

Problem

Check if brackets are balanced
Complexity: O(n) time, O(n) space

Key Insight

Stack to match closing with opening
Map closing → opening for cleaner code

```java
public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    Map<Character, Character> pairs = Map.of(
        ')', '(', '}', '{', ']', '['
    );

    for (char c : s.toCharArray()) {
        if (!pairs.containsKey(c)) {
            stack.push(c);  // Opening
        } else {
            if (stack.isEmpty() ||
                stack.pop() != pairs.get(c)) {
                return false;
            }
        }
    }

    return stack.isEmpty();
}
```

Common Mistakes

× Don't forget final `stack.isEmpty()`
× Check `isEmpty()` BEFORE `pop()`
✓ Map closing to opening

5. Group Anagrams

Problem

Group strings that are anagrams
Complexity: O(n × k) time, O(n × k) space

Key Insight

Use character count as key (faster than sorting)
`Arrays.toString(count)` simplest for key

```java
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> groups = new HashMap<>();

    for (String s : strs) {
        int[] count = new int[26];
        for (char c : s.toCharArray()) {
            count[c - 'a']++;
        }
        String key = Arrays.toString(count);

        groups.putIfAbsent(key, new ArrayList<>());
        groups.get(key).add(s);
    }

    return new ArrayList<>(groups.values());
}
```

6. Longest Substring Without Repeating

Problem

Longest substring with unique characters
Complexity: O(n) time, O(min(n,m)) space

Key Insight

Sliding window + HashMap for last seen index
MUST use Math.max for left pointer!

```java
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int left = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);

        if (lastSeen.containsKey(c)) {
            left = Math.max(left, lastSeen.get(c) + 1);
        }

        lastSeen.put(c, right);
        maxLen = Math.max(maxLen, right - left + 1);
    }

    return maxLen;
}
```

WHY Math.max?

Example: s = "abba"
When right=3 (second 'a'):
• lastSeen['a'] = 0
• But left already at 2
• Can't move backwards!
× left = 0 + 1 = 1 (WRONG)
✓ left = Math.max(2, 1) = 2

7. Minimum Window Substring

Problem

Smallest window in s containing all chars of t
Complexity: O(n+m) time, O(k) space

Key Insight

Expand until valid, then shrink to minimize
Track unique chars matched, not total count
need = countT.size() (unique), not sum

```java
public String minWindow(String s, String t) {
    if (t.isEmpty()) return "";

    Map<Character, Integer> countT = new HashMap<>();
    for (char c : t.toCharArray()) {
        countT.put(c, countT.getOrDefault(c, 0) + 1);
    }

    Map<Character, Integer> window = new HashMap<>();
    int have = 0, need = countT.size();
    int[] res = {-1, -1};
    int resLen = Integer.MAX_VALUE;
    int l = 0;

    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        window.put(c, window.getOrDefault(c, 0) + 1);

        if (countT.containsKey(c) &&
            window.get(c).equals(countT.get(c))) {
            have++;
        }

        while (have == need) {
            if ((r - l + 1) < resLen) {
                resLen = r - l + 1;
                res[0] = l; res[1] = r;
            }

            char leftChar = s.charAt(l);
            window.put(leftChar, window.get(leftChar) - 1);
            if (countT.containsKey(leftChar) &&
                window.get(leftChar) < countT.get(leftChar)) {
                have--;
            }
            l++;
        }
    }

    return resLen == Integer.MAX_VALUE ? "" :
           s.substring(res[0], res[1] + 1);
}
```

Common Mistakes

× Use `.equals()` for Integer comparison
× Track unique chars, not total count
✓ need = countT.size()

8. Longest Repeating Character Replacement

Problem

Longest substring after replacing $\leq$ k chars
Complexity: O(n) time, O(1) space

Key Insight

Window valid if (windowSize - maxFreq) $\leq$ k
maxCount NEVER decreases (optimization!)

```java
public int characterReplacement(String s, int k) {
    int[] freq = new int[26];
    int left = 0, maxCount = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        freq[s.charAt(right) - 'A']++;
        maxCount = Math.max(maxCount,
                            freq[s.charAt(right) - 'A']);

        while ((right - left + 1) - maxCount > k) {
            freq[s.charAt(left) - 'A']--;
            left++;
        }

        maxLen = Math.max(maxLen, right - left + 1);
    }

    return maxLen;
}
```

Why maxCount Doesn't Decrease?

Only care about finding BETTER windows
If maxCount was 5, we've seen window with 5
Only want windows with MORE than 5
Keeping maxCount high = stricter condition
= only find better windows

9. Maximum Average Subarray

Problem

Max average of k consecutive elements
Complexity: O(n) time, O(1) space

Key Insight

Fixed-size sliding window
Slide: remove left, add right

```java
public double findMaxAverage(int[] nums, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += nums[i];
    }

    int maxSum = sum;

    for (int i = k; i < nums.length; i++) {
        sum = sum - nums[i - k] + nums[i];
        maxSum = Math.max(maxSum, sum);
    }

    return (double) maxSum / k;
}
```

Common Mistakes

× Don't recalculate sum O(n×k)
✓ Slide window O(n)
✓ Return double, not int

10. Container With Most Water

Problem

Maximum area between two lines
Complexity: O(n) time, O(1) space

Key Insight

Move pointer of shorter height
Water BETWEEN lines, not on top
Area = min(height[l], height[r]) × (r - l)

```java
public int maxArea(int[] height) {
    int l = 0, r = height.length - 1;
    int maxArea = 0;

    while (l < r) {
        int h = Math.min(height[l], height[r]);
        int area = h * (r - l);
        maxArea = Math.max(maxArea, area);

        if (height[l] < height[r]) {
            l++;
        } else {
            r--;
        }
    }

    return maxArea;
}
```

NOT Trapping Rain Water!

× Don't use leftMax/rightMax
✓ Use height[l], height[r] directly
✓ Water BETWEEN lines

11. Trapping Rain Water

Problem

Calculate trapped rainwater on top of bars
Complexity: O(n) time, O(1) space

Key Insight

Water level = min(leftMax, rightMax) - height
Move pointer FIRST, then calculate
All in SAME iteration!

```java
public int trap(int[] height) {
    if (height == null || height.length == 0)
        return 0;

    int l = 0, r = height.length - 1;
    int leftMax = height[l], rightMax = height[r];
    int res = 0;

    while (l < r) {
        if (leftMax < rightMax) {
            l++;  // Move FIRST
            leftMax = Math.max(leftMax, height[l]);
            res += leftMax - height[l];
        } else {
            r--;  // Move FIRST
            rightMax = Math.max(rightMax, height[r]);
            res += rightMax - height[r];
        }
    }

    return res;
}
```

Order of Operations

```
if (leftMax < rightMax) {
l++;          // Step 1
leftMax = ...    // Step 2
res += ...       // Step 3
}
```

All in SAME iteration!

12. Find Min in Rotated Sorted Array

Problem

Find minimum in rotated sorted array
Complexity: O(log n) time, O(1) space

Key Insight

Compare mid with right
Early exit: if nums[left] < nums[right]

```java
public int findMin(int[] nums) {
    int left = 0, right = nums.length - 1;

    if (nums[left] < nums[right])
        return nums[left];

    while (left < right) {  // <, not <=
        int mid = left + (right - left) / 2;

        if (nums[mid] > nums[right]) {
            left = mid + 1;
        } else {
            right = mid;  // Keep mid
        }
    }

    return nums[left];
}
```

Common Mistakes

✓ Use `left < right`
✓ Use `right = mid`
✓ Compare with nums[right]
× NOT nums[left]

13. Search in Rotated Sorted Array

Problem

Search target in rotated sorted array
Complexity: O(log n) time, O(1) space

Key Insight

Determine which half is sorted
Check if target in sorted half

```java
public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {  // <=, not <
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) return mid;

        if (nums[left] <= nums[mid]) {
            // Left half sorted
            if (nums[left] <= target &&
                target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            // Right half sorted
            if (nums[mid] < target &&
                target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }

    return -1;
}
```

Common Mistakes

✓ Use `left <= right`
✓ Use `mid - 1`, `mid + 1`
✓ Check: nums[left] <= nums[mid]
✓ Must check target in sorted half

14. Maximum Subarray (Kadane's)

Problem

Find subarray with maximum sum
Complexity: O(n) time, O(1) space

Key Insight

At each position: extend or start fresh
If cumulative sum negative, start fresh

```java
public int maxSubArray(int[] nums) {
    int currMax = nums[0];
    int maxSum = nums[0];

    for (int i = 1; i < nums.length; i++) {
        currMax = Math.max(nums[i],
                           currMax + nums[i]);
        maxSum = Math.max(maxSum, currMax);
    }

    return maxSum;
}
```

Why It Works

At position i, two choices:
1. Extend: currMax + nums[i]
2. Start fresh: nums[i]

Choose larger! If previous sum negative,
it drags us down → start fresh

Code Templates

Variable Sliding Window

```java
public ReturnType slidingWindow(Type[] arr) {
    Map<Type, Integer> window = new HashMap<>();
    int left = 0;
    ReturnType result = initialValue;

    for (int right = 0; right < arr.length; right++) {
        // Expand
        window.put(arr[right],
            window.getOrDefault(arr[right], 0) + 1);

        // Shrink if violated
        while (conditionViolated) {
            window.put(arr[left],
                window.get(arr[left]) - 1);
            if (window.get(arr[left]) == 0) {
                window.remove(arr[left]);
            }
            left++;
        }

        // Update result
        result = update(result, right - left + 1);
    }

    return result;
}
```

Fixed Sliding Window

```java
public ReturnType fixedWindow(int[] arr, int k) {
    // First window
    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += arr[i];
    }

    ReturnType result = sum;

    // Slide
    for (int i = k; i < arr.length; i++) {
        sum = sum - arr[i - k] + arr[i];
        result = update(result, sum);
    }

    return result;
}
```

Two Pointers (Opposite)

```java
public ReturnType twoPointers(int[] arr) {
    int left = 0, right = arr.length - 1;
    ReturnType result = initialValue;

    while (left < right) {
        Type current = calculate(arr[left], arr[right]);
        result = update(result, current);

        if (shouldMoveLeft) {
            left++;
        } else {
            right--;
        }
    }

    return result;
}
```

Binary Search (Position)

```java
public int binarySearchPosition(int[] arr) {
    int left = 0, right = arr.length - 1;

    while (left < right) {  // <
        int mid = left + (right - left) / 2;

        if (condition(arr[mid])) {
            left = mid + 1;
        } else {
            right = mid;  // Keep mid
        }
    }

    return left;
}
```

Binary Search (Value)

```java
public int binarySearchValue(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {  // <=
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}
```

HashMap Frequency

```java
public ReturnType frequencyCount(Type[] arr) {
    Map<Type, Integer> freq = new HashMap<>();

    // Count
    for (Type item : arr) {
        freq.put(item,
            freq.getOrDefault(item, 0) + 1);
    }

    // Process
    for (Map.Entry<Type, Integer> entry :
         freq.entrySet()) {
        Type key = entry.getKey();
        int count = entry.getValue();
        // ...
    }

    return result;
}
```

Common Mistakes

Integer Comparison

```java
// ✗ WRONG
if (map.get(key) == map.get(other))

// ✓ CORRECT
if (map.get(key).equals(map.get(other)))
```

Result Update Timing

```java
// ✗ WRONG - outside while
while (violated) { shrink(); }
result = update();

// ✓ CORRECT - inside for
for (int right = 0; right < n; right++) {
    expand();
    while (violated) { shrink(); }
    result = update();
}
```

Array Bounds

```java
// ✗ WRONG
left++;
process(arr[left]);  // May be OOB!

// ✓ CORRECT
left++;
if (left < arr.length) {
    process(arr[left]);
}
```

Binary Search Loop

```java
// ✗ WRONG - infinite loop
while (left < right) {
    int mid = (left + right) / 2;
    if (cond) left = mid;  // Stuck!
    else right = mid - 1;
}

// ✓ CORRECT
while (left < right) {
    int mid = left + (right - left) / 2;
    if (cond) left = mid + 1;
    else right = mid;
}
```

Substring End

```
// ✗ WRONG - excludes end
s.substring(start, end);

// ✓ CORRECT - includes end
s.substring(start, end + 1);
```

Stack Empty Check

```
// ✗ WRONG - crashes if empty
if (stack.pop() != expected)

// ✓ CORRECT
if (stack.isEmpty() || stack.pop() != expected)
```

Max/Min Init

```
// ✗ WRONG - fails for negatives
int maxSum = 0;

// ✓ CORRECT
int maxSum = Integer.MIN_VALUE;
// OR
int maxSum = arr[0];
```

Key Insights

Container vs Trapping

Container With Most Water:
- height[l], height[r] directly
- Water BETWEEN lines
- Move shorter height

Trapping Rain Water:
- leftMax, rightMax (running)
- Water ON TOP of bars
- Move smaller max

Math.max in Sliding Window

`left = Math.max(left, lastSeen[c] + 1)`
Why? lastSeen[c] might be before current left
Example: "abba", can't move backwards!

maxCount Never Decreases

In Character Replacement:
Only care about BETTER windows
If maxCount was 5, only want > 5
Keeping high = stricter = better windows

HashMap vs Array

Use Array if:
✓ Limited charset (ASCII/lowercase)
✓ Known value range
✓ Need O(1) space
Use HashMap if:
✓ Unicode/arbitrary chars
✓ Flexible requirements

Quick Reference

Complexity Table

Problem	Time	Space
Product Except Self	O(n)	O(1)
Subarray Sum = K	O(n)	O(n)
Top K Frequent	O(n)	O(n)
Valid Parentheses	O(n)	O(n)
Group Anagrams	O(n×k)	O(n×k)
Longest Substring	O(n)	O(m)
Min Window	O(n+m)	O(k)
Char Replacement	O(n)	O(1)
Max Average	O(n)	O(1)
Container Water	O(n)	O(1)
Trap Rain Water	O(n)	O(1)
Find Min Rotated	O(log n)	O(1)
Search Rotated	O(log n)	O(1)
Max Subarray	O(n)	O(1)

Pattern Recognition

Keyword	Pattern
Contiguous subarray	Sliding Window
All pairs	Two Pointers
Sorted/rotated	Binary Search
Matching/balanced	Stack
Frequency/count	HashMap
Top K	Heap/Bucket Sort
Max/Min subarray	Kadane's/DP

Data Structure Choice

Need	Use
Matching pairs	Stack
Frequency counting	HashMap
Last seen index	HashMap
Prefix sums	HashMap
Char frequency	Array[26/128]
Order matters	ArrayList

Learning Progression

1. **Master Patterns:**
   - Sliding Window (4 problems)
   - Two Pointers (2 problems)
   - Binary Search (2 variants)
2. **HashMap Techniques:**
   - Frequency counting
   - Prefix sums
   - Last seen tracking
3. **Array Manipulation:**
   - Prefix/suffix products
   - Bucket sort
4. **Stack Applications:**
   - Matching brackets
   - Monotonic stack
5. **Dynamic Programming:**
   - Kadane's algorithm
   - State optimization

Daily Practice

1. Solve without looking (30 min)
2. Compare with optimal (10 min)
3. Dry run with example (15 min)
4. Explain WHY it works (5 min)
5. Code from memory (10 min)

Before Interview

☐ Recognize patterns from keywords?
☐ Know < vs <= in binary search?
☐ Explain sliding window?
☐ Remember Math.max() for left?
☐ Differentiate Container vs Trapping?
☐ Initialize max/min correctly?
☐ Check isEmpty() before pop()?
☐ Optimize HashMap to Array?
☐ Understand complexity?
☐ Explain WHY algorithm works?

Good luck! ⋆ You've got this! ⋆